AWS re:Invent

CMP307

Deep Dive on Amazon EC2 Instances & Performance Optimization Best Practices

Mark Duffield

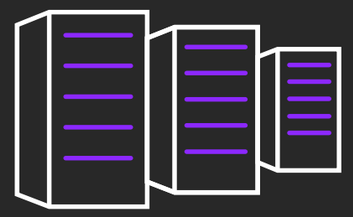
Worldwide Tech Lead, Semiconductors
Amazon Web Services

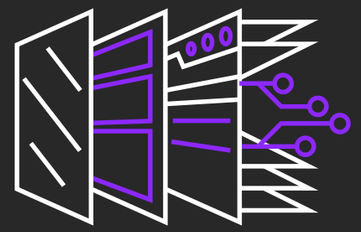
AWS re:Invent

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

aws

Amazon Elastic Compute Cloud (Amazon EC2)





Infrastructure

Regions
AZs
Data centers

Instances

Characteristics
Choices
Hypervisors
Bare metal

Performance

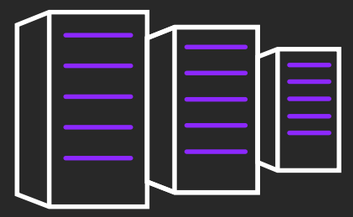
AMI/OS
Threads
Clocksource
Processor State
Xen spinlock
NUMA control
User Limits
Instance Store
Network

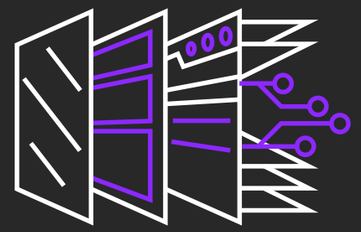
Tools

lstopo (hwloc)
turbostat
htop
nethogs
perf
iperf3

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon EC2 deep dive





Infrastructure

Regions
AZs
Data centers

Instances

Characteristics
Choices
Hypervisors
Bare metal

Performance

AMI/OS
Threads
Clocksource
Processor State
Xen spinlock
NUMA control
User Limits
Instance Store
Network

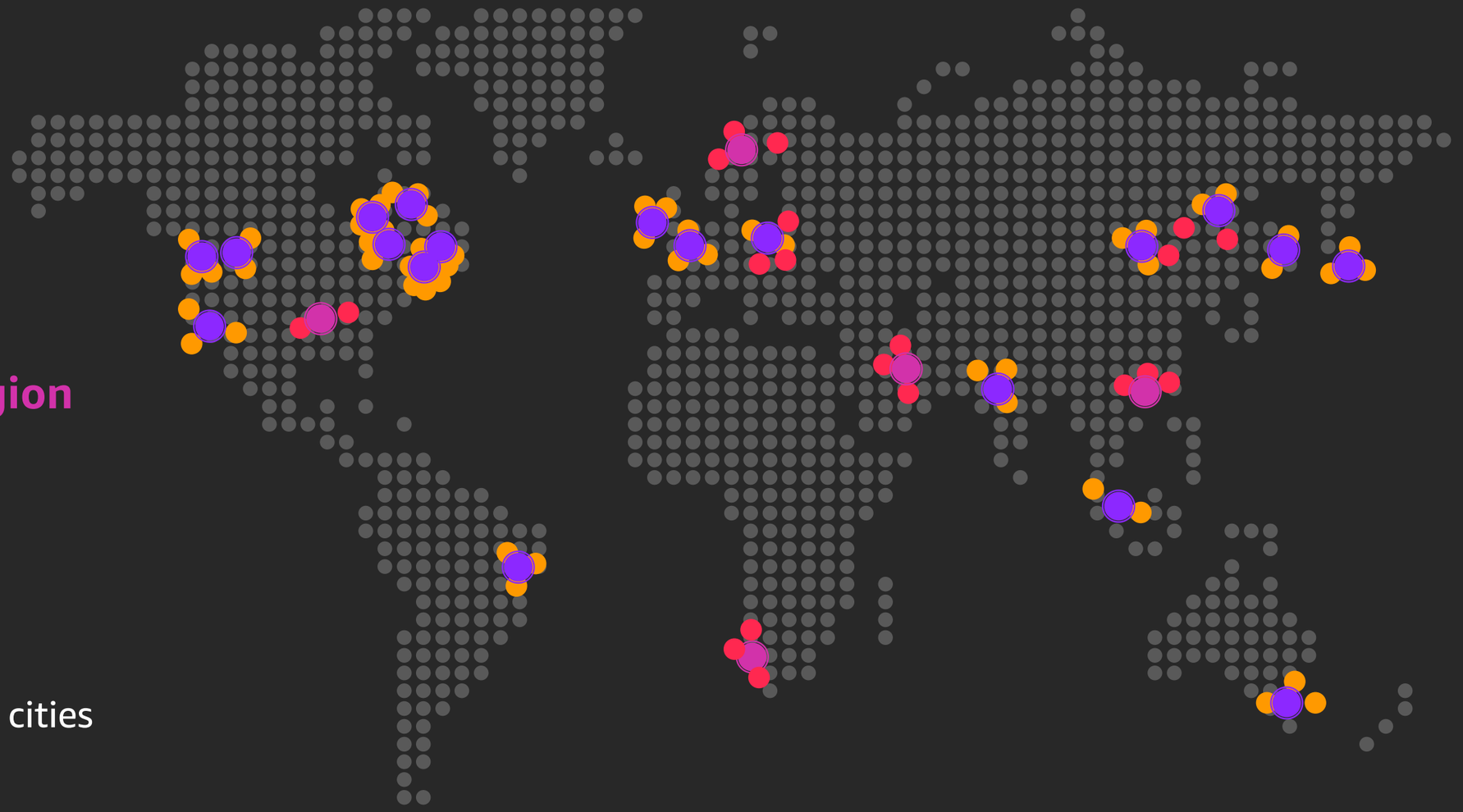
Tools

lstopo (hwloc)
turbostat
htop
nethogs
perf
iperf3

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Global compute platform for compute everywhere

- Regions
- Availability Zones
- New Regions
- New Availability Zones



55 Availability Zones
18 Regions + 1 Local Region
Coming soon
15 New Availability Zones
5 New regions

Global edge network
138 Points of presence
11 Regional edge caches in 62 cities across 29 countries

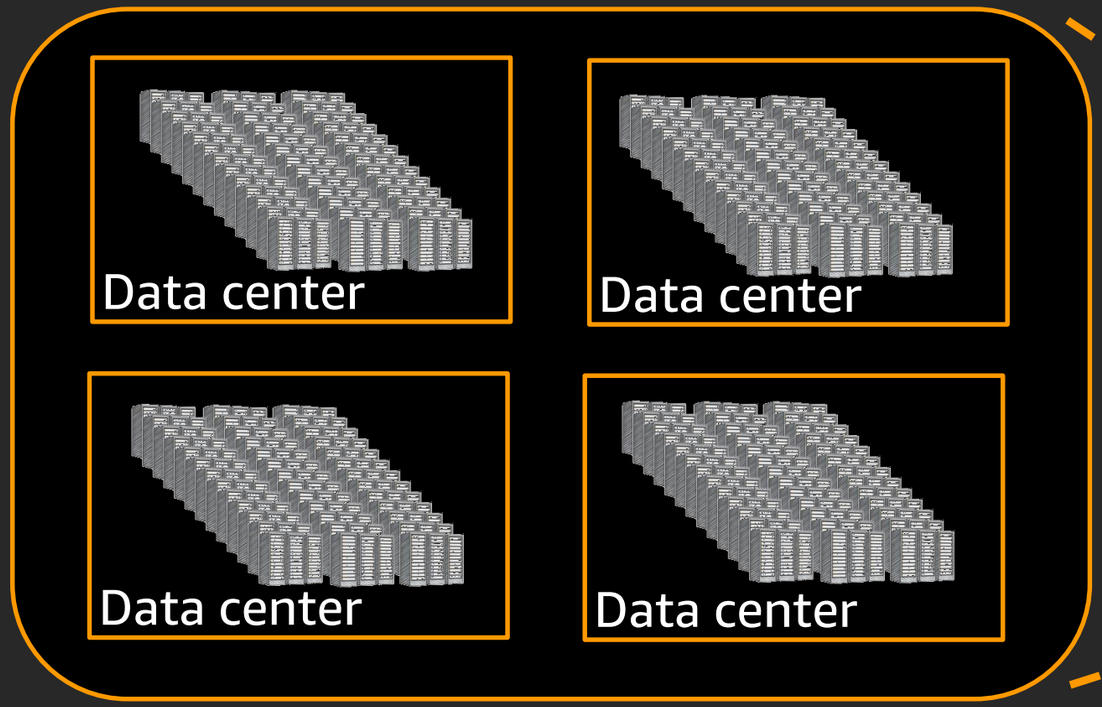
AWS re:Invent

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

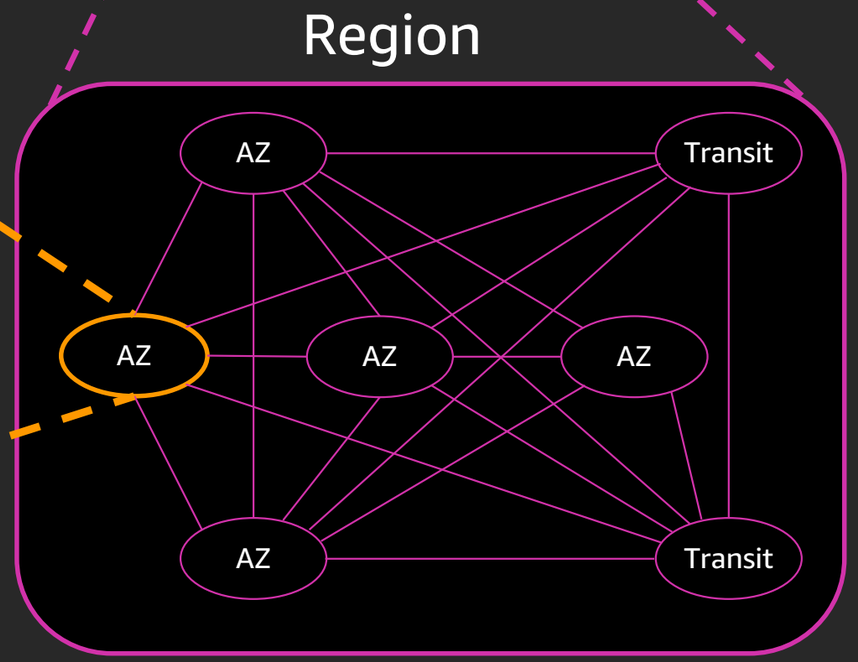
aws

Example AWS Availability Zone

- All regions have two or more AZs, some as many as six
- Each AZ is one or more data centers
  - No data center is in two AZs

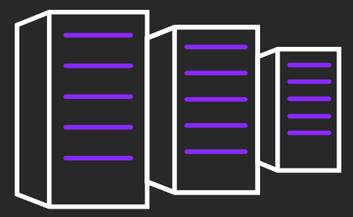


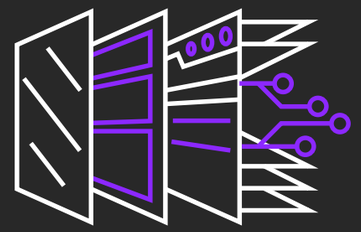
Availability Zone



© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon EC2 deep dive





Infrastructure

Regions
AZs
Data centers

Instances

Characteristics
Choices
Hypervisors
Bare metal

Performance

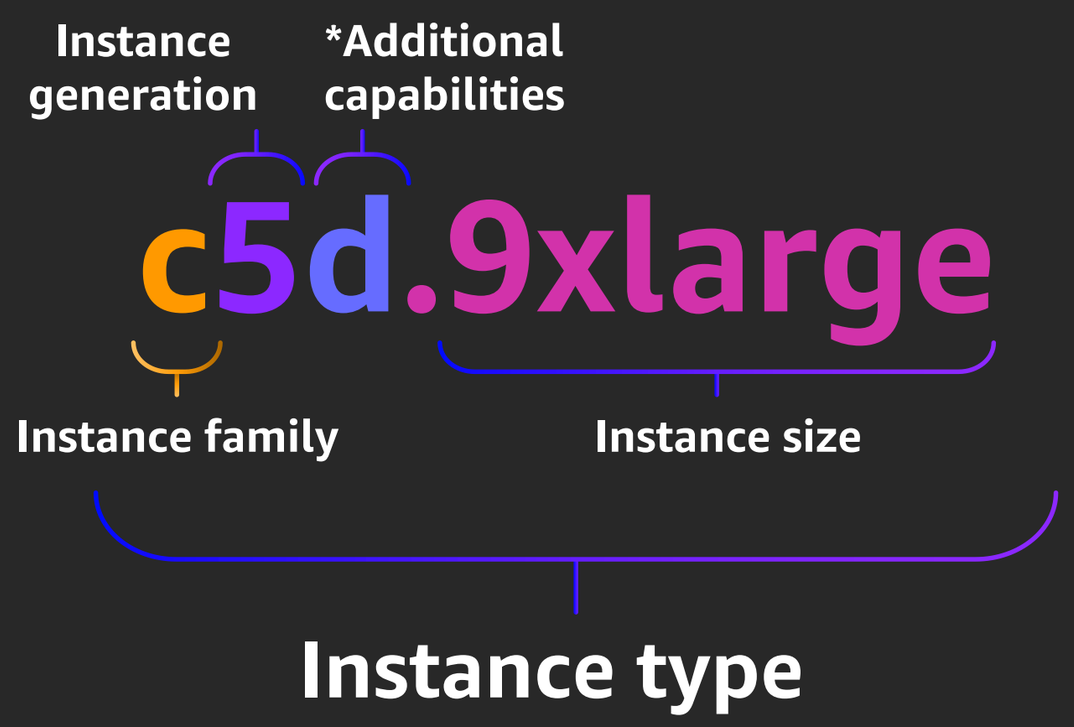
AMI/OS
Threads
Clocksource
Processor State
Xen spinlock
NUMA control
User Limits
Instance Store
Network

Tools

lstopo (hwloc)
turbostat
htop
nethogs
perf
iperf3

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon EC2 instance characteristics



Instance sizes are comprised of compute, memory, storage, and network

Hypervisor options

- Xen (older instances)
- KVM (Nitro Hypervisor)
- No hypervisor (AWS Nitro System)

*Not on all instances, and also not used on older instances (e.g., c3)

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Broadest and deepest platform choice

Workloads

General purpose

Burstable

Compute intensive

Memory intensive

Storage (High I/O)

Dense storage

GPU compute

Graphics intensive

+

Capabilities

NEW! Processor
(ARM, Intel, AMD)

Fast processors
(up to 4.0 GHz)

High memory footprint
(up to 12 TiB)

Instance storage
(HDD and NVMe options)

Accelerated computing
(GPUs and FPGA)

NEW! Networking
(up to 100 Gbps)

Bare metal

Size
(Nano, micro, small, medium, large, xlarge, 2xlarge)

+

Options

Elastic Graphics

Elastic Block Storage

=

175

instance types

Virtually any workload and business need

AWS re:Invent

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

aws

Broadest choice of processors and architectures

Intel Xeon Scalable (Skylake) processor

NVDIA Tesla V100 GPUs

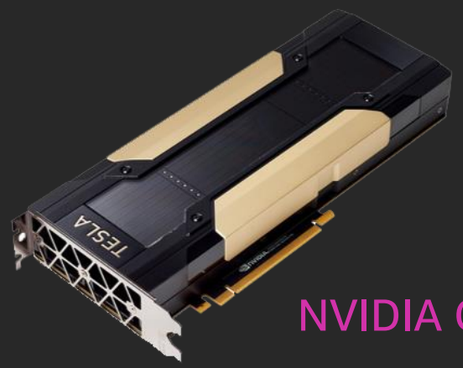
NEW! AMD EPYC processor

NEW! Amazon Cloud Processor for Arm-based applications

Right compute for the right application and workload

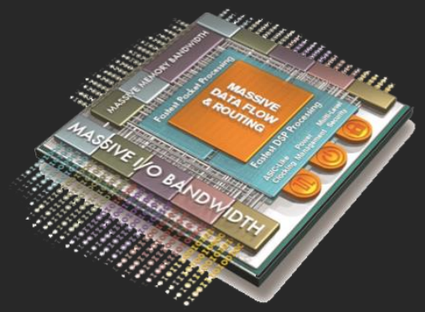
© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

GPU and FPGA for accelerated computing



NVIDIA GPU

Xilinx UltraScale + FPGA



P2/P3: GPU-accelerated computing

Enabling a high degree of parallelism—Each GPU has thousands of cores

Consistent, well documented set of APIs (CUDA, OpenACC, OpenCL)

Supported by a wide variety of ISVs and open source frameworks

F1: FPGA-accelerated computing

Massively parallel—Each FPGA includes millions of parallel system logic cells

Flexible—No fixed instruction set, can implement wide or narrow datapaths

Programmable using available, cloud-based FPGA development tools

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Which hypervisor do we use?

Old: **Xen**

- Original hypervisor
- Consumed excessive resources
- Limited optimization

New (Nov/2017): **Custom KVM based hypervisor**

- Nitro instances
- Less server resources used, more resources for the customer
- AWS optimized

Bare metal: **No AWS provided hypervisor**

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

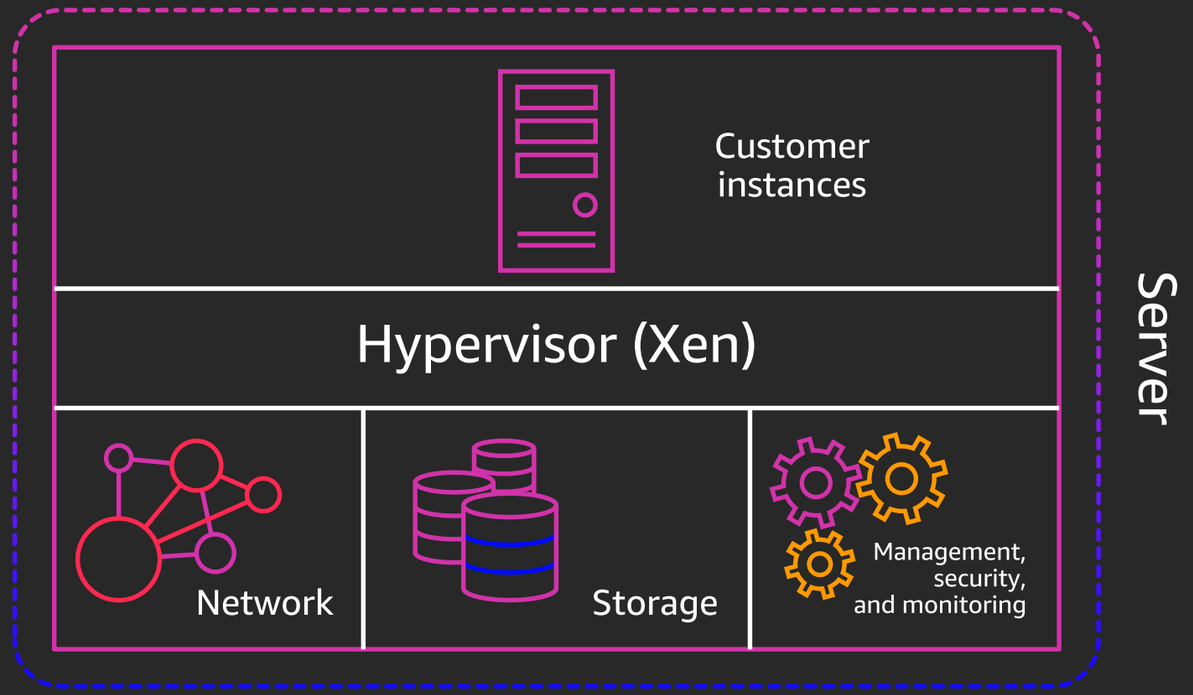
Hypervisor update

Original EC2 host architecture

All resources were on the server

Instance goals

- Security
- Performance
- Familiarity

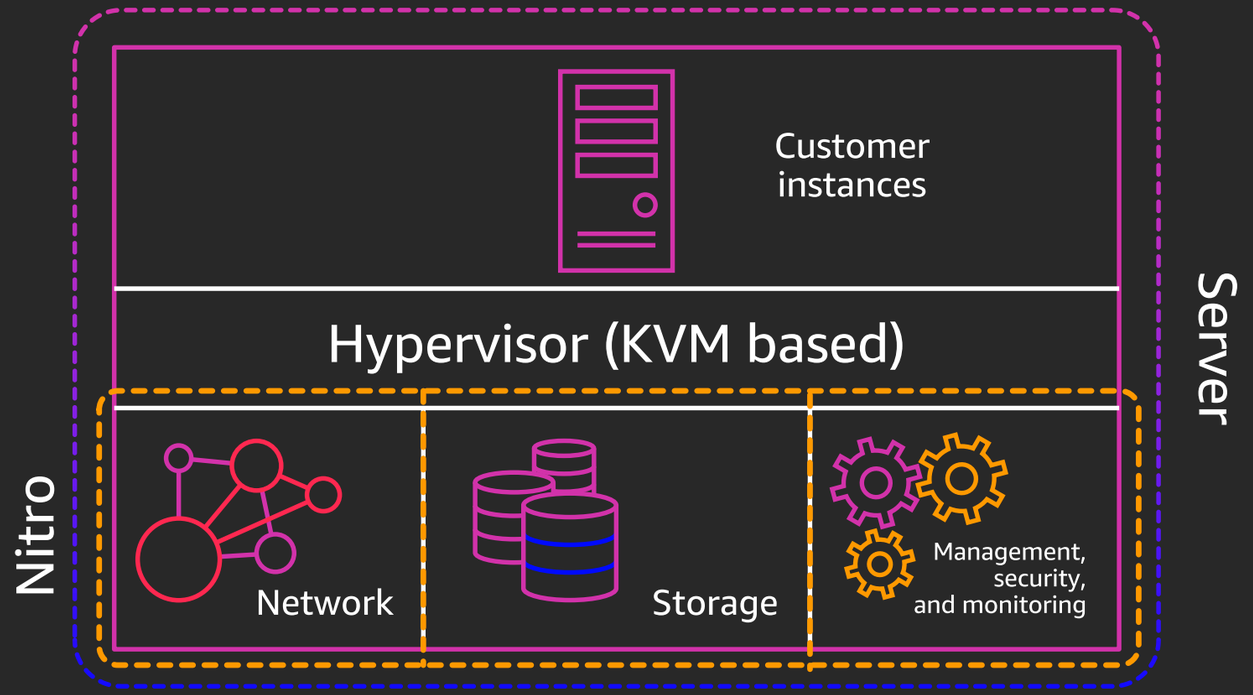


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

EC2 instance built on AWS Nitro System

Nearly 100% of available compute resources available to customers' workload

Improved security

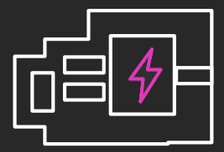


powered by annapurnalabs an amazon company

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

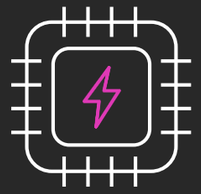
Innovation enabled by AWS Nitro System

Nitro Card



Local NVMe storage
Amazon EBS
Networking, monitoring, and security

Nitro Security Chip



Integrated into motherboard
Protects hardware resources

Nitro Hypervisor

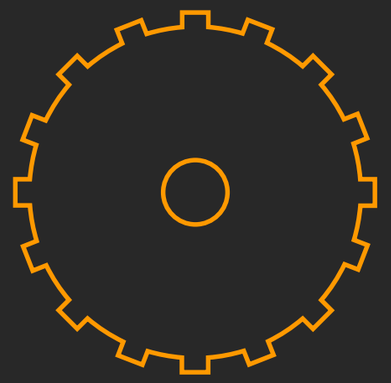
Lightweight hypervisor
Memory and CPU allocation
Bare metal-like performance

Modular building blocks for rapid design and delivery of EC2 instances

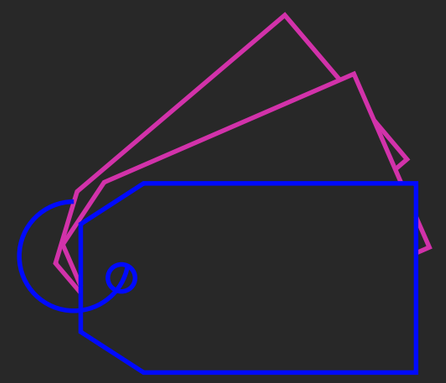
© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

EC2 bare metal—No AWS provided hypervisor

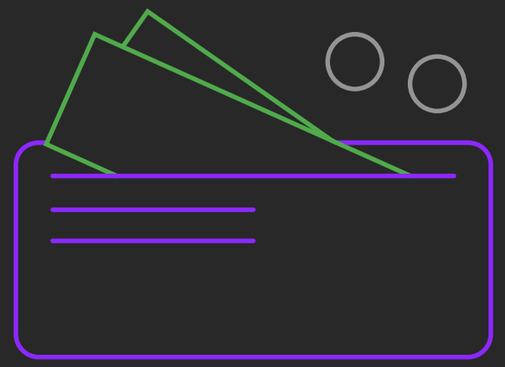
Direct hardware access with the all the benefits of cloud computing



Non virtualized workloads



Hypervisor specific workloads



Workloads with restricted licensing

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

C5 Instances—Intel® Xeon® Scalable Processor

Intel Skylake
@ 3.0 GHz (turbo to 3.5GHz)

Supports AVX512

C-state controls

Nitro System, a combination of dedicated hardware and lightweight hypervisor

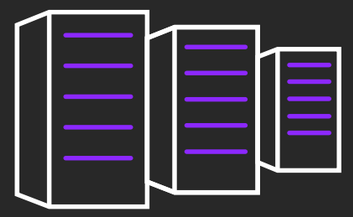
Up to 25 Gbps network

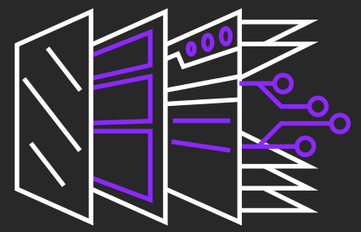
C4	C5
36 vCPUs	72 vCPUs
“Haswell”	“Skylake”
4 Gbps to EBS	14 Gbps to EBS
60 GiB memory	144 GiB memory

AVX 512

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon EC2 deep dive





Infrastructure

Regions
AZs
Data centers

Instances

Characteristics
Choices
Hypervisors
Bare metal

Performance

AMI/OS
Threads
Clocksource
Processor State
Xen spinlock
NUMA control
User Limits
Instance Store
Network

Tools

lstopo (hwloc)
turbostat
htop
nethogs
perf
iperf3

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

“Launching new instances and running tests in parallel is easy…[when choosing an instance] **there is no substitute for measuring the performance of your full application.”**

—EC2 documentation

What is an Amazon Machine Image (AMI)?

Provides the information required to launch an instance

Launch multiple instances from a single AMI

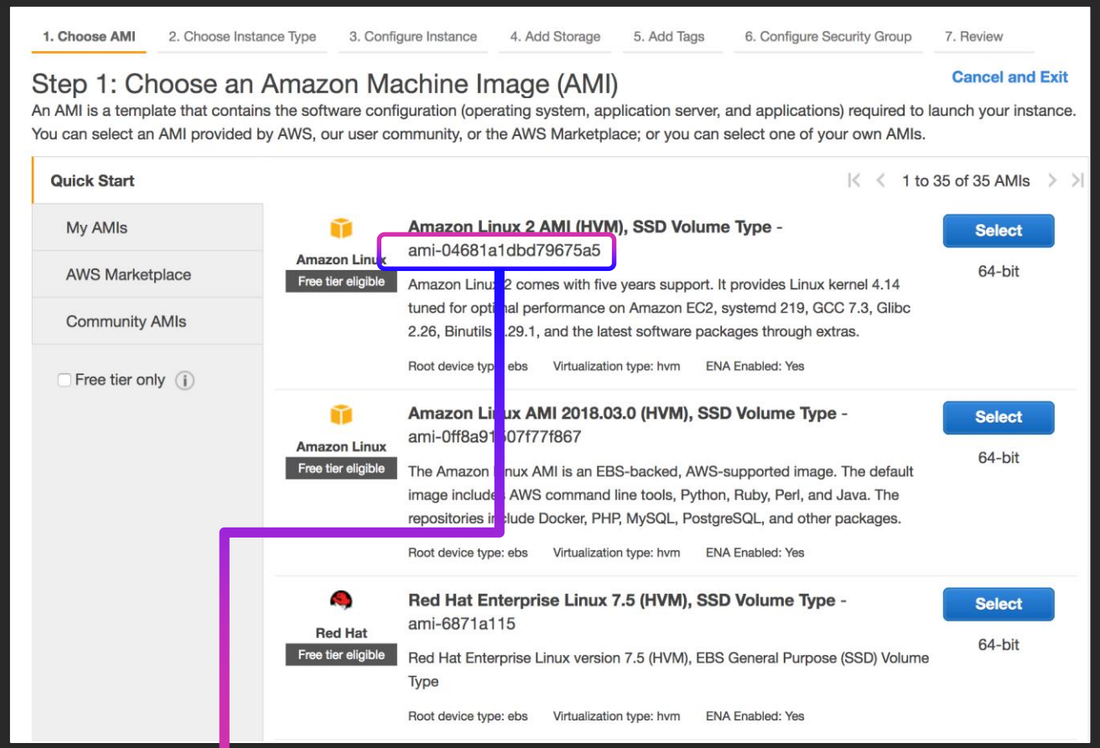
An AMI includes the following (and probably more)

- A template for the root volume (for example, operating system, applications)
- Launch permissions that control which AWS accounts can use the AMI
- Block device mapping that specifies volumes to attach to the instance

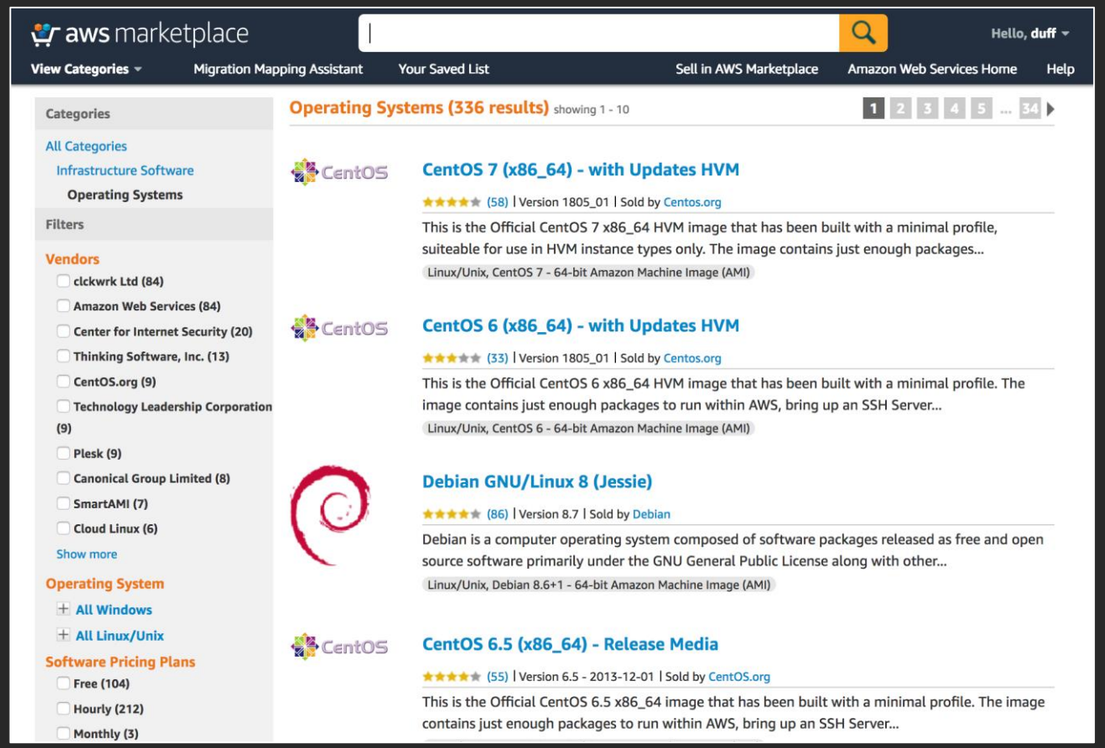
© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Choosing an AMI

AWS Console



AWS Marketplace



Use the AMI ID to launch through the API or AWS Command Line Interface (AWS CLI)

```
aws ec2 run-instances --image-id ami-04681a1dbd79675a5 --instance-type c4.8xlarge --count 10 --key-name MyKey
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Choosing the right AMI and OS

Choose latest OS level your tool or application supports

Kernel should be at 3.10 or higher

- As much as a 40% performance improvement
- **Should not be using a 2.6** or older kernel

Minimum recommended OS*

- The most recent version of **Amazon Linux 2 or Amazon Linux AMI**
- **Ubuntu version 16.04** or latest LTS release provided by AWS
- **Red Hat Enterprise Linux version 7.4**
- **CentOS 7 version 1708_11**
- **SUSE Linux Enterprise Server 12 SP2**
- **FreeBSD 11.1** or later (does not support F1 instances)

*Includes NVMe kernel module

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ssd-instance-store.html#nvme-ssd-volumes

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon Linux 2

Enterprise ready

5 years of LTS

Ongoing security & maintenance updates

Robust partner ecosystem

Innovation included

Optimized for AWS

Modern tooling and packages

Amazon Linux Extras repository

Universal availability

AMI for Amazon EC2 use

Docker container images

Virtual machines

No cost

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AMI and OS on Nitro instances

ENA installed (latest version) and AMI enabled

Before launching a Nitro instance, the operating system will need to have the ENA driver installed and the ENA flag on the AMI will need to be set as well

NVMe installed (latest version) – Amazon EBS volumes on Nitro

Amazon EBS volumes are exposed as **NVMe block devices** on Nitro instances. The device names are /dev/nvme0n1, /dev/nvme1n1, and so on.

You will need NVMe drivers to boot with Nitro based instance types

Options

Option 1 (less work): Use existing AMI with necessary config (in other words, ENA and NVMe)

Option 2 (more work): Use a Xen based AMI and update config

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Nitro check—OS modules

Web search for “c5_m5_checks_script.sh”

```
$ sudo ./c5_m5_checks_script.sh
-----------------------------------------------
OK     NVMe Module is installed and available on your instance
OK     ENA Module with version 1.5.0g is installed and available on your instance
OK     fstab file looks fine and does not contain any device names
-----------------------------------------------
```

https://aws.amazon.com/premiumsupport/knowledge-center/boot-error-linux-m5-c5/

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Nitro Check—ENA on AMI

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/enhanced-networking-ena.html

```
$ aws ec2 describe-instances --instance-ids <inst_id> --query "Reservations[].Instances[].EnaSupport"
[
    true
]
```

If the above command is not true, install ENA OS and enable ENA. See ENA AWS documentation

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/enhanced-networking-ena.html

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Multiple threads per core

A vCPU is a thread on a x86 physical core

Divide by two to get total number of physical cores

Can be a concern for CPU heavy applications

Control threads three examples

1. Without reboot on a running system
2. With CPU Options (awscli)
3. Kernel line, persistent

Use ‘lscpu’ to validate layout

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Control threads* 1/3

On a running system

```
$ for cpunum in $(cat /sys/devices/system/cpu/cpu*/topology/thread_siblings_list | \
cut -s -d, -f2- | tr '-' '\n' | tr ',' ‘\n’ | sort -un); do
    echo 0 | sudo tee /sys/devices/system/cpu/cpu${cpunum}/online
done
```

*Test with and without

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Control threads* 2/3

At launch with **CPU Options, either AWS CLI or AWS Console**

```
$ aws ec2 run-instances --image-id ami-asdfasdfasdfasdf --instance-type z1d.12xlarge
  --cpu-options "CoreCount=24,ThreadsPerCore=1” --key-name My_Key_Name
```

To verify the CpuOptions were set, use **describe-instances**

```
$ aws ec2 describe-instances --instance-ids i-1234qwer1234qwer
...
"CpuOptions": {
     "CoreCount": 24,
     "ThreadsPerCore": 1
},
...
```

*Test with and without

AWS re:Invent

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

aws

Control threads* 3/3

At the kernel line

Add “maxcpus” to the kernel line in the /etc/default/grub file and rebuild boot file

```
GRUB_CMDLINE_LINUX_DEFAULT="console=tty0 ... nvme_core.io_timeout=4294967295 maxcpus=24 ”
```

Verify maxcpus was set

```
$ cat /proc/cmdline
root=LABEL=/ console=tty1 console=ttyS0 maxcpus=24 xen_nopvspin=1
```

*Test with and without

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Verify threads

Before disabling multiple threads per core

```
$ lscpu --extended
CPU NODE SOCKET CORE L1d:L1i:L2:L3 ONLINE
0   0    0      0    0:0:0:0       yes
1   0    0      1    1:1:1:0       yes
2   0    0      0    0:0:0:0       yes
3   0    0      1    1:1:1:0       yes
```

After disabling multiple threads per core

```
$ lscpu --extended
CPU NODE SOCKET CORE L1d:L1i:L2:L3 ONLINE
0   0    0      0    0:0:0:0       yes
1   0    0      1    1:1:1:0       yes
2   -    -      -    :::           no
3   -    -      -    :::           no
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Clocksource

Xen based instances default is Xen pvclock (in the hypervisor)

- Avoid communication with the hypervisor and use the CPU clock
- Set clocksource to tsc

Nitro instances use kvm-clock clocksource

- The default kvm-clock clocksource on Nitro based instance types provides similar performance benefits as tsc on previous-generation Xen based instances.

- Instances with AMD processors use the Nitro system (no need change the clocksource)

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Time intensive application

```
#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <time.h>

#define BILLION 1E9

int main(){
    float diff_ns;
    struct timespec start, end;
    int x;
    clock_gettime(CLOCK_MONOTONIC, &start);
        for ( x = 0; x < 100000000; x++ ) {
            struct timeval tv;
            gettimeofday(&tv, NULL);
        }
    clock_gettime(CLOCK_MONOTONIC, &end);
    diff_ns = (BILLION * (end.tv_sec - start.tv_sec)) + (end.tv_nsec - start.tv_nsec);
    printf ("Elapsed time is %.4f seconds\n", diff_ns / BILLION );
    return 0;
}
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Using Xen pvclock for clocksource

```
$ strace -c ./test
Elapsed time is 10.0336 seconds
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 99.99    3.322956           2   2001862           gettimeofday
  0.00    0.000096           6        16           mmap
  0.00    0.000050           5        10           mprotect
  0.00    0.000038           8         5           open
  0.00    0.000026           5         5           fstat
  0.00    0.000025           5         5           close
  0.00    0.000023           6         4           read
  0.00    0.000008           8         1         1 access
  0.00    0.000006           6         1           brk
  0.00    0.000006           6         1           execve
  0.00    0.000005           5         1           arch_prctl
  0.00    0.000000           0         1           munmap
------ ----------- ----------- --------- --------- ----------------
100.00    3.323239               2001912         1 total
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Change clocksource Xen based instance

Set the clocksource to tsc at the command line:

```
$ sudo su -c "echo tsc > /sys/devices/system/cl*/cl*/current_clocksource"
```

Or at the kernel command (e.g. /etc/default/grub):

clocksource=tsc

Verify that the clocksource is set to tsc:

```
$ cat /sys/devices/system/cl*/cl*/current_clocksource
tsc
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Using TSC as clocksource

```
$ strace -c ./test
Elapsed time is 2.0787 seconds
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 32.97    0.000121           7        17           mmap
 20.98    0.000077           8        10           mprotect
 11.72    0.000043           9         5           open
 10.08    0.000037           7         5           close
  7.36    0.000027           5         6           fstat
  6.81    0.000025           6         4           read
  2.72    0.000010          10         1           munmap
  2.18    0.000008           8         1         1 access
  1.91    0.000007           7         1           execve
  1.63    0.000006           6         1           brk
  1.63    0.000006           6         1           arch_prctl
  0.00    0.000000           0         1           write
------ ----------- ----------- --------- --------- ----------------
100.00    0.000367                    53         1 total
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Processor state control

Which instances? You'll need at least a socket on an Intel instance

C-state

Entering deeper idle states, allows active cores to achieve higher clock frequencies, **but deeper idle states require more time to exit,** may not be appropriate for latency-sensitive workloads, Windows: no options to control c states

P-state (not on Nitro instances)

Controls the CPU's ability to change frequency, including enabling or disabling Turbo boost

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/processor_state_control.html

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Processor state control

C-state

Linux: limit c-state by adding “`intel_idle.max_cstate=1`” to kernel command line

P-state (not on Nitro instances) – set no_turbo

```
$ sudo sh -c "echo 1 > /sys/devices/system/cpu/intel_pstate/no_turbo“
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

P-state and C-state defaults

```
[ec2-user ~]$ sudo turbostat stress -c 2 -t 10
stress: info: [30680] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [30680] successful run completed in 10s
pk cor CPU    %c0  GHz  TSC SMI    %c1    %c3    %c6    %c7   %pc2   %pc3   %pc6   %pc7  Pkg_W RAM_W PKG_% RAM_%
             5.54 3.44 2.90   0   9.18   0.00  85.28   0.00   0.00   0.00   0.00   0.00  94.04 32.70 54.18  0.00
 0   0   0   0.12 3.26 2.90   0   3.61   0.00  96.27   0.00   0.00   0.00   0.00   0.00  48.12 18.88 26.02  0.00
 0   0  18   0.12 3.26 2.90   0   3.61
 0   1   1   0.12 3.26 2.90   0   4.11   0.00  95.77   0.00
 0   1  19   0.13 3.27 2.90   0   4.11
 0   2   2   0.13 3.28 2.90   0   4.45   0.00  95.42   0.00
 0   2  20   0.11 3.27 2.90   0   4.47
 0   3   3   0.05 3.42 2.90   0  99.91   0.00   0.05   0.00
 0   3  21  97.84 3.45 2.90   0   2.11
...
 1   1  10   0.06 3.33 2.90   0  99.88   0.01   0.06   0.00
 1   1  28  97.61 3.44 2.90   0   2.32
...
10.002556 sec
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

P-state = no_turbo, and C-state = 1

```
[ec2-user ~]$ sudo turbostat stress -c 2 -t 10
stress: info: [5389] dispatching hogs: 2 cpu, 0 io, 0 vm, 0 hdd
stress: info: [5389] successful run completed in 10s
pk cor CPU    %c0  GHz  TSC SMI    %c1    %c3    %c6    %c7   %pc2   %pc3   %pc6   %pc7  Pkg_W RAM_W PKG_% RAM_%
             5.59 2.90 2.90   0  94.41   0.00   0.00   0.00   0.00   0.00   0.00   0.00 128.48 33.54 200.00  0.00
 0   0   0   0.04 2.90 2.90   0  99.96   0.00   0.00   0.00   0.00   0.00   0.00   0.00  65.33 19.02 100.00  0.00
 0   0  18   0.04 2.90 2.90   0  99.96
 0   1   1   0.05 2.90 2.90   0  99.95   0.00   0.00   0.00
 0   1  19   0.04 2.90 2.90   0  99.96
 0   2   2   0.04 2.90 2.90   0  99.96   0.00   0.00   0.00
 0   2  20   0.04 2.90 2.90   0  99.96
 0   3   3   0.05 2.90 2.90   0  99.95   0.00   0.00   0.00
 0   3  21  99.95 2.90 2.90   0   0.05
...
 1   1  28  99.92 2.90 2.90   0   0.08
 1   2  11   0.06 2.90 2.90   0  99.94   0.00   0.00   0.00
 1   2  29   0.05 2.90 2.90   0  99.95
```

No turbo and cores not active are in the C1 C-state, ready to accept instructions

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Xen spinlock

Most OS distributions use a paravirtualized spinlock implementation optimized for oversubscribed Xen virtual machines. Disable unless you are running on burstable T2 instances (T3 uses Nitro, kvm based hypervisor)

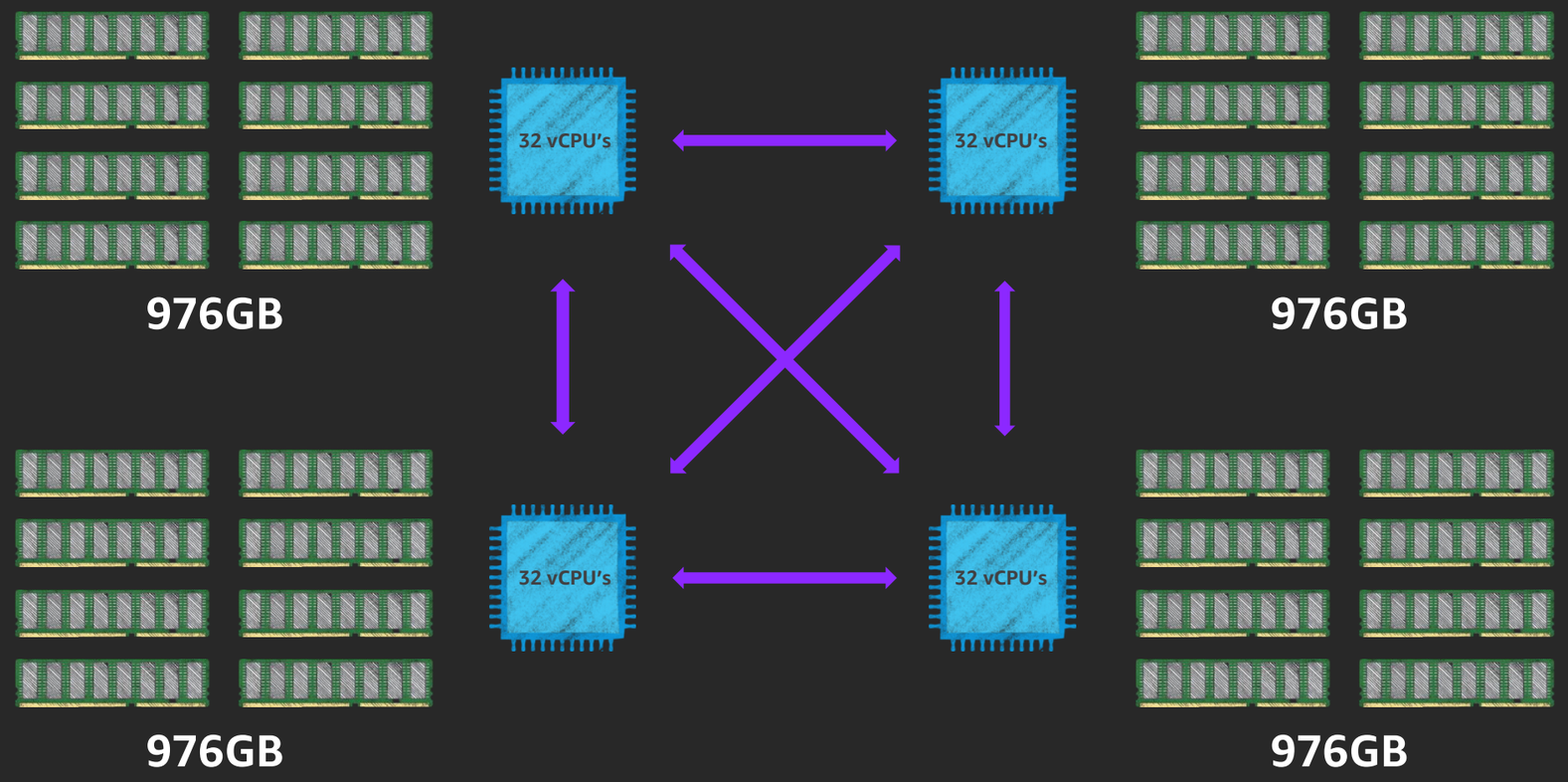
Can be expensive from a performance perspective causes the VM to slow down when running multithreaded with locks

Use the xen_nopvspin=1 grub setting to get closer to bare-metal locking

```
kernel /boot/vmlinuz-4.4.41-36.55.amzn1.x86_64 ... selinux=0 xen_nopvspin=1

$ dmesg | grep spinlocks
[    0.000000] xen: PV spinlocks disabled
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

NUMA controls



© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

NUMA controls

Find zones with "`lscpu | grep NUMA`"

Does your app have more memory that fits in a single socket?

- Linux: set "numa=off" in grub to disable NUMA awareness

Do you have many processes or a footprint less than a single socket?

- Linux: use "numactl" to restrict them to specific cores or nodes
- Examples:

```
$ numactl --cpunodebind=0 --membind=0 ./a.out        # bind to node
$ numactl --physcpubind=+0-15 --membind=0 ./a.out    # bind to cpus
```

Windows: Use processor affinity to lock applications to specific cores

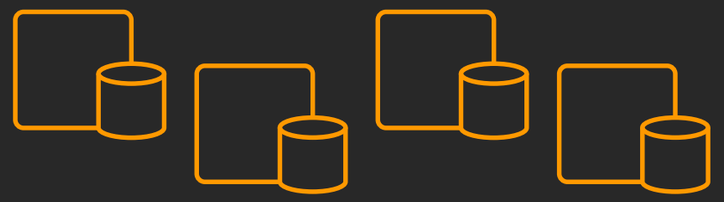
© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

User limits

```
# core file size (blocks, -c)
*          hard    core          0
*          soft    core          0

# file size (blocks, -f)
*          hard    fsize         unlimited
*          soft    fsize         unlimited

# stack size (kbytes, -s)
*          hard    stack         unlimited
*          soft    stack         unlimited

# max user processes (-u
*          soft    nproc         16384
*          hard    nproc         16384
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Instance store



Temporary block-level storage

Physically attached to host computer

Lifetime

Data lost when

- Drive failure
- Instance stops
- Instance terminates

Data persists on reboot

Instance store data loss prevention

- Create RAID 1/5/6
- Move data to Amazon S3 or EBS
- Create a fault tolerant FS

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Instance store—NVMe

I3 instances

- Up to 8 NVMe volumes locally attached that can achieve up to 16 GiB/s and over 3M IOPS

Instance types with "d" option (for example, c5d, m5d, z1d)

Encryption

Usage

- Build your own file servers
- Cache for file system solutions (for example, ZFS)
- Local scratch space

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS network

AWS proprietary network, 10Gbps, 25Gbps, and 100Gbps

- Highest performance in largest EC2 instance sizes
- Cluster placement groups, high speed, low latency network fabric, no network oversubscription

Enhanced networking

- Nearly 3 million PPS, reduced instance-to-instance latencies, more consistent network performance

Amazon EC2 to Amazon Simple Storage Service (Amazon S3)

- Up to 25 Gbps of bandwidth using multiple streams

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Network performance

Use Cluster Placement Groups

Tune MTU, use jumbo packets per application requirement

Use multiple elastic network interfaces

- For example, one interface for the application and the other file system mounts

Manually distribute packet receive interrupts

Set up Receive Packet Steering (RPS)

- At software level, direct packets to specific CPUs

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/enhanced-networking-os.html
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/network_mtu.html

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Enhanced network for HPC and machine learning

Up to 100 Gbps network bandwidth

NEW!

C5n

Fastest compute for high performance workloads

NEW!

P3dn

Fastest machine learning training in the cloud

+

NEW!

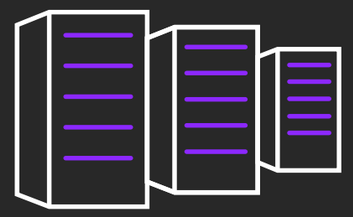
Elastic Fabric Adapter for HPC

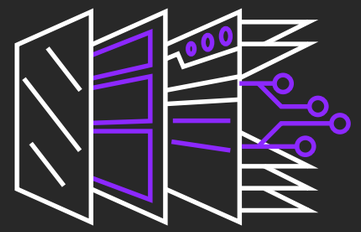
Best for large HPC workloads

https://aws.amazon.com/blogs/aws/new-c5n-instances-with-100-gbps-networking/

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Amazon EC2 deep dive





Infrastructure

Regions
AZs
Data centers

Instances

Characteristics
Choices
Hypervisors
Bare metal

Performance

AMI/OS
Threads
Clocksource
Processor State
Xen spinlock
NUMA control
User Limits
Instance Store
Network

Tools

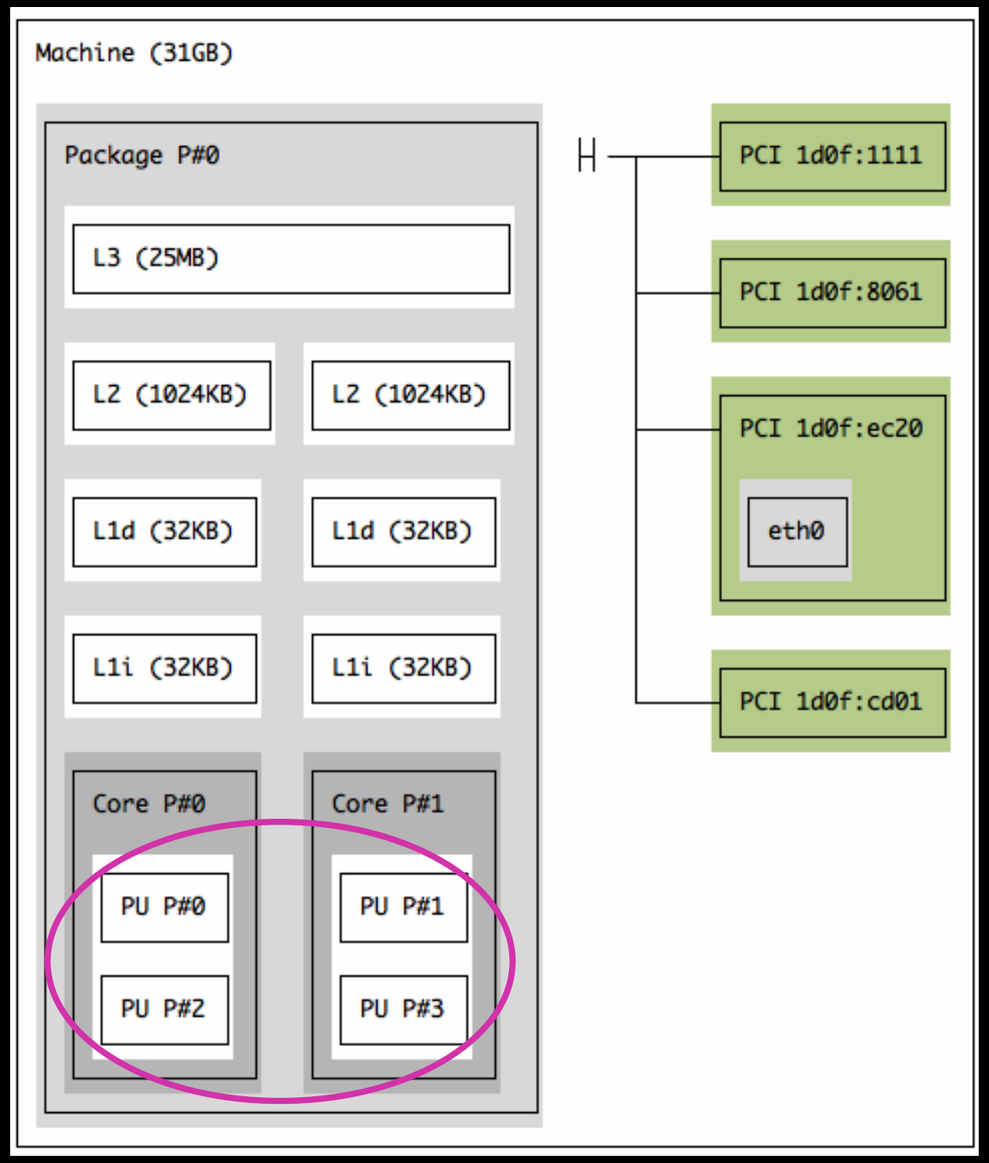
lstopo (hwloc)
turbostat
htop
nethogs
perf
iperf3

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

lstopo (hwloc)

```
$ lstopo-no-graphics --of ascii --rect
```

z1d.xlarge



Another way to check threads

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

turbostat—Monitor CPU (gives accurate frequency)

‘lscpu’ or ’cat /proc/cpuinfo’ may report the base frequency

```
$ sudo turbostat
Core     CPU      Avg_MHz  Busy%    Bzy_MHz  TSC_MHz  IRQ      SMI      CPU%c1   CPU%c6   PkgWatt  RAMWatt
-        -        4000     100.00   4000     3400     10089    0        0.00     0.00     0.00     0.00
0        0        4000     100.00   4000     3400     1253     0        0.00     0.00     0.00     0.00
0        4        4000     100.00   4000     3400     1252     0        0.00
1        1        4000     100.00   4000     3400     1261     0        0.00     0.00
1        5        4000     100.00   4000     3400     1256     0        0.00
2        2        4000     100.00   4000     3400     1276     0        0.00     0.00
2        6        4000     100.00   4000     3400     1277     0        0.00
3        3        4000     100.00   4000     3400     1258     0        0.00     0.00
3        7        4000     100.00   4000     3400     1256     0        0.00
```

Shown for z1d

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

htop—Monitor CPU (stress with no threads)

```
1  [||||||||||||||||||||||||||||||100.0%]   7  [||||||||||||||||||||||||||||||100.0%]   13 [||||||||||||||||||||||||||||||100.0%]   19 [||||||||||||||||||||||||||||||100.0%]
2  [||||||||||||||||||||||||||||||100.0%]   8  [||||||||||||||||||||||||||||||100.0%]   14 [||||||||||||||||||||||||||||||100.0%]   20 [||||||||||||||||||||||||||||||100.0%]
3  [||||||||||||||||||||||||||||||100.0%]   9  [||||||||||||||||||||||||||||||100.0%]   15 [||||||||||||||||||||||||||||||100.0%]   21 [||||||||||||||||||||||||||||||100.0%]
4  [||||||||||||||||||||||||||||||100.0%]   10 [||||||||||||||||||||||||||||||100.0%]   16 [||||||||||||||||||||||||||||||100.0%]   22 [||||||||||||||||||||||||||||||100.0%]
5  [||||||||||||||||||||||||||||||100.0%]   11 [||||||||||||||||||||||||||||||100.0%]   17 [||||||||||||||||||||||||||||||100.0%]   23 [||||||||||||||||||||||||||||||100.0%]
6  [||||||||||||||||||||||||||||||100.0%]   12 [||||||||||||||||||||||||||||||100.0%]   18 [||||||||||||||||||||||||||||||100.0%]   24 [||||||||||||||||||||||||||||||100.0%]
Mem[|||                                                                                  Tasks: 62, 67 thr; 25 running
Swp[                                                                                     Load average:       9.25 4.79
                                                                                         Uptime: 44 days, 17:08:41
```

PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
10935	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.87	stress -c 24 -t 60
10928	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.85	stress -c 24 -t 60
10929	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.85	stress -c 24 -t 60
10932	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.85	stress -c 24 -t 60
10925	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.84	stress -c 24 -t 60
10921	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.82	stress -c 24 -t 60
10922	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.82	stress -c 24 -t 60
10915	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.81	stress -c 24 -t 60
10926	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.84	stress -c 24 -t 60
10916	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.81	stress -c 24 -t 60
10933	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.85	stress -c 24 -t 60
10919	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.82	stress -c 24 -t 60
10913	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.80	stress -c 24 -t 60
10930	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.85	stress -c 24 -t 60
10920	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.82	stress -c 24 -t 60
10923	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.82	stress -c 24 -t 60
10936	ec2-user	20	0	7316	96	0	R	100.	0.0	0:48.82	stress -c 24 -t 60
10931	ec2-user	20	0	7316	96	0	R	99.0	0.0	0:48.85	stress -c 24 -t 60
10934	ec2-user	20	0	7316	96	0	R	99.0	0.0	0:48.85	stress -c 24 -t 60
10924	ec2-user	20	0	7316	96	0	R	99.0	0.0	0:48.82	stress -c 24 -t 60
10917	ec2-user	20	0	7316	96	0	R	99.0	0.0	0:48.81	stress -c 24 -t 60
10914	ec2-user	20	0	7316	96	0	R	99.0	0.0	0:48.80	stress -c 24 -t 60
10918	ec2-user	20	0	7316	96	0	R	99.0	0.0	0:48.81	stress -c 24 -t 60
10927	ec2-user	20	0	7316	96	0	R	99.0	0.0	0:48.60	stress -c 24 -t 60
10911	ec2-user	20	0	120M	4780	2920	R	0.0	0.0	0:00.59	htop

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

nethogs—Monitor network traffic

```
$ sudo nethogs
```

```
NetHogs version 0.8.5

    PID USER     PROGRAM                                  DEV           SENT      RECEIVED
    977 ec2-us.. /usr/bin/python2                         eth0       1052.800  200054.016 KB/sec
    817 ec2-us.. sshd: ec2-user@pts/0                     eth0        130.690      49.471 KB/sec
      ? root     unknown TCP                                          0.000       0.000 KB/sec

    TOTAL                                                            1183.490  200103.486 KB/sec
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

perf—Linux profiling with performance counters

```
[ec2-user@RHEL7 ~]$ sudo perf stat ./ebizzy-0.3/ebizzy -S 10
425,143 records/s
real 10.00 s
user 397.28 s
sys   0.18 s

Performance counter stats for './ebizzy-0.3/ebizzy -S 10':

     397515.862535      task-clock (msec)         #   39.681 CPUs utilized
            25,256      context-switches          #    0.064 K/sec
             2,201      cpu-migrations            #    0.006 K/sec
            14,109      page-faults               #    0.035 K/sec

      10.017856000 seconds time elapsed
```


© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

iperf3—Test network throughput

```
[ ID] Interval           Transfer     Bandwidth       Retr
[  4]   0.00-120.00 sec  34.3 GBytes  2.46 Gbits/sec   33             sender
[  4]   0.00-120.00 sec  34.3 GBytes  2.46 Gbits/sec                  receiver
[  6]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec   20             sender
[  6]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec                  receiver
[  8]   0.00-120.00 sec  34.3 GBytes  2.46 Gbits/sec   22             sender
[  8]   0.00-120.00 sec  34.3 GBytes  2.46 Gbits/sec                  receiver
[ 10]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec   10             sender
[ 10]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec                  receiver
[ 12]   0.00-120.00 sec  34.3 GBytes  2.46 Gbits/sec    8             sender
[ 12]   0.00-120.00 sec  34.3 GBytes  2.46 Gbits/sec                  receiver
[ 14]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec   19             sender
[ 14]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec                  receiver
[ 16]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec   18             sender
[ 16]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec                  receiver
[ 18]   0.00-120.00 sec  34.3 GBytes  2.46 Gbits/sec   15             sender
[ 18]   0.00-120.00 sec  34.3 GBytes  2.46 Gbits/sec                  receiver
[ 20]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec   18             sender
[ 20]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec                  receiver
[ 22]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec   15             sender
[ 22]   0.00-120.00 sec  34.3 GBytes  2.45 Gbits/sec                  receiver
[SUM]   0.00-120.00 sec   343 GBytes  24.5 Gbits/sec  178              sender
[SUM]   0.00-120.00 sec   343 GBytes  24.5 Gbits/sec                   receiver

iperf Done.
```

https://aws.amazon.com/premiumsupport/knowledge-center/network-throughput-benchmark-linux-ec2/

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Additional information

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

AWS News Blog

https://aws.amazon.com/blogs/aws/

Menu | aws | Contact Sales | Products | Solutions | Pricing | More | My Account | Sign Up

Blog Home | Category | Edition | Follow | Search Blogs

AWS News Blog

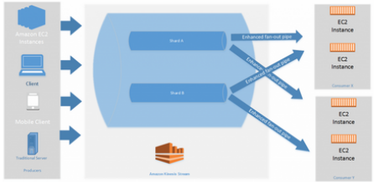
Name	vCPUs	Baseline Performance / vCPU	Memory	Price / Hour (Linux)	Price / Hour (Windows)
t3.nano	2	5%	0.5 GiB	$0.0052	$0.0980
t3.micro	2	10%	1 GiB	$0.0104	$0.0196
t3.small	2	20%	2 GiB	$0.0209	$0.0393
t3.medium	2	20%	4 GiB	$0.0418	$0.0602
t3.large	2	30%	8 GiB	$0.0835	$0.1111
t3.xlarge	4	40%	16 GiB	$0.1670	$0.2406
t3.2xlarge	8	40%	32 GiB	$0.3341	$0.4813

New T3 Instances – Burstable, Cost-Effective Performance

by Jeff Barr | on 21 AUG 2018 | in Amazon EC2, Launch, News | Permalink | Comments | Share

We launched the t1.micro instance type in 2010, and followed up with the first of the T2 instances (micro, small, and medium) in 2014, more sizes in 2015 (nano) and 2016 (xlarge and 2xlarge), and unlimited bursting last year. Today we are launching T3 instances in twelve regions. These general-purpose instances are even more cost-effective [...]

Read More



Amazon Kinesis Data Streams Adds Enhanced Fan-Out and HTTP/2 for Faster Streaming

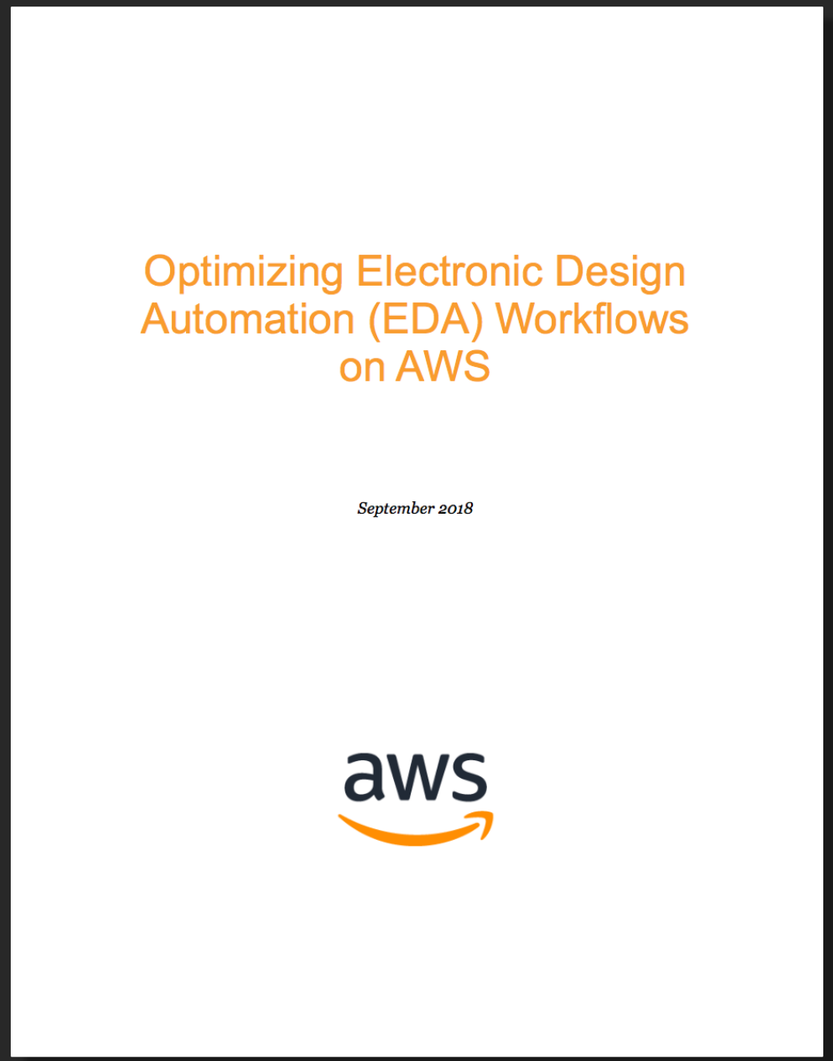
by Randall Hunt | on 21 AUG 2018 | in Amazon Kinesis, Analytics, AWS Big Data | Permalink | Comments | Share

We launched a new feature for Amazon Kinesis Data Streams (KDS) called Enhanced Fan-out, which enables developers to take advantage of a new low latency HTTP/2 data retrieval API. This new API allows customers to attach more applications to a single data stream while maintaining read performance across all of the applications consuming the stream. The HTTP/2 interface reduces the latency between ingestion of data and consumption of data by a consumer application by about 75%, down to less than 50 milliseconds.

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

EDA White Paper

bit.ly/aws-eda-whitepaper

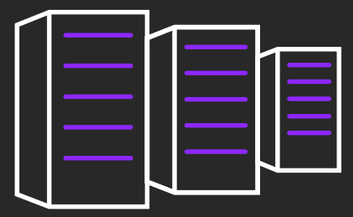


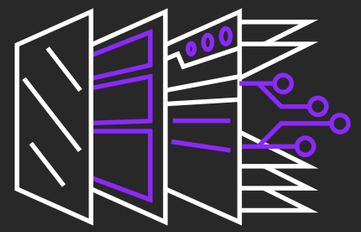
AWS re:Invent

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

aws

Amazon EC2 deep dive





Infrastructure

Regions
AZs
Data centers

Instances

Characteristics
Choices
Hypervisors
Bare metal

Performance

AMI/OS
Threads
Clocksource
Processor State
Xen spinlock
NUMA control
User Limits
Instance Store
Network

Tools

lstopo (hwloc)
turbostat
htop
nethogs
perf
iperf3

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Resources

Operating System Optimizations

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/enhanced-networking-os.html

AMI/OS info

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ssd-instance-store.html#nvme-ssd-volumes

Nitro Check—OS Modules

https://aws.amazon.com/premiumsupport/knowledge-center/boot-error-linux-m5-c5/

Nitro Check—ENA on AMI

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/enhanced-networking-ena.html

Processor State

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/processor_state_control.html

iperf3 testing

https://aws.amazon.com/premiumsupport/knowledge-center/network-throughput-benchmark-linux-ec2/

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Resources

Network Tuning

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/enhanced-networking-os.html
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/network_mtu.html

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Tomorrow night!

re:PLAY

SPONSORED BY:

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

Questions?

AWS re:Invent

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

aws

Thank you!

Mark Duffield
duff@amazon.com

AWS re:Invent

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.

aws

Please complete the session survey in the mobile app.

© 2018, Amazon Web Services, Inc. or its affiliates. All rights reserved.